```
class PredictiveConvHebbian(nn.Module):
    def __init__(self, in_ch, out_ch, kernel_size):
        super().__init__()
        self.conv = nn.Conv2d(in_ch, out_ch,
kernel_size, padding=1)

    def hebbian_update(self, x, pred_error,
lr=0.01):
        # pred_error = x_next - conv(x)
        batch, ch, h, w = x.shape

        for b in range(batch):
            for o in range(self.conv.out_channels):
                for i in range(self.conv.in_channels):
                    # Local patches
                    patches =
extract_patches(x[b:b+1, i],
self.conv.kernel_size)
                    delta_k = lr * (pred_error[b:b+1, o]
* patches).mean(dim=[2,3])
                    self.conv.weight.data[o, i] +=
delta_k
```